

Asynchronous programming



Future, async and await

Eng | Ruqaih Hussein Salman

2027-2028

Table of Contents

Complete lecture structure

Section	Coverage	Page
1	Async mental model, isolate, event loop, responsive UI	2
2	Why asynchronous programming matters: I/O vs CPU work	3
3	Synchronous vs asynchronous execution and completion order	4
4	Future<T>, Future.delayed, callbacks, and completion outcomes	5–6
5	async/await, return types, exceptions, and try/catch/finally	7
6	Timeout, bounded retry, and sequential vs Future.wait concurrency	8
7	StatefulWidget loading, mounted safety, and duplicate actions	9–10
8	FutureBuilder, AsyncSnapshot, Future ownership, refresh, retry	11–12
9	Streams, async*, yield, subscriptions, and cancellation	13–14
10	StreamBuilder and a stored real-time clock stream	15
11	Complete practical FutureBuilder and StreamBuilder apps	16–19
12	Integrated checklist, assessment, answer key, references	20–23

Learning Outcomes and Study Map

What the student should master by the end of the lecture

Area	Student mastery
Mental model	Explain isolate execution, queued events, asynchronous I/O waiting, and responsive Flutter frames.
Execution order	Predict synchronous print order and later completion of Future-based work.
Future	Create and consume Future<T> values; distinguish uncompleted, value, and error completion.
Reliability	Handle exceptions, timeout, cleanup, retry, and the fact that timeout is not cancellation.
Concurrency	Choose sequential await or Future.wait according to data dependencies.
Lifecycle	Check mounted/context.mounted after asynchronous gaps before using State or BuildContext.
FutureBuilder	Store the Future before build and render loading, data, and error branches safely.
Streams	Create repeated values, use StreamBuilder, and cancel manually owned subscriptions.

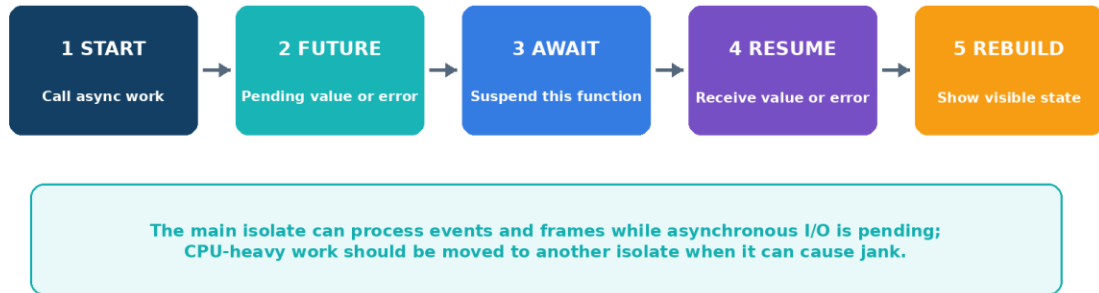
LEARNING STRATEGY

- Predict the console order or visible state before running each example.
- At every await, identify which async function pauses and what resumes after completion.
- When an async UI fails, inspect the Future/Stream source, error path, lifecycle, rebuild trigger, and visible feedback.

1. Async Programming Mental Model

One isolate, queued work, and a responsive Flutter frame

From a slow operation to responsive UI



CORE IDEA

- Each Dart isolate has its own memory and one thread running an event loop.
- Asynchronous I/O can wait without blocking that isolate from processing other queued work.
- `async/await` does not automatically move CPU-heavy computation to another isolate.

Dart event loop mental model



One isolate executes one Dart instruction at a time; asynchronous waiting is concurrency, not automatic CPU parallelism.

Term	Meaning	Diagnostic question
Call stack	Synchronous functions currently executing.	Which function is running now?
Future	One eventual value or error.	What type will complete?
Microtask queue	Higher-priority queued callbacks processed before the next ordinary event turn.	Should this run before a timer/event?
Event queue	Timers, I/O completions, pointer events, and other events.	What event resumes the work?
Flutter frame	Framework scheduling for build/layout/paint.	Can the UI keep rendering while I/O waits?

COMMON MISTAKES

- Calling asynchronous I/O “parallel CPU execution.”
- Adding `async` to a CPU-intensive loop and expecting the UI to stay smooth.
- Blocking the main isolate with synchronous parsing, image processing, or file work.

2. Why Asynchronous Programming Matters

Waiting is common; freezing the user interface is not acceptable

LECTURE 8 · 1

ASYNC FOUNDATIONS — THE WAITING MODEL

Understand the control flow before syntax: Future, await, success, error, cleanup.

30-SECOND QUESTION

What happens when Flutter starts work that will finish later?



THE EXECUTION ORDER YOU SHOULD SEE



IF YOU REMEMBER ONLY FOUR THINGS

Future = one eventual result

value OR error

async

function returns a Future

await

wait here without freezing UI

try / catch / finally

success · error · cleanup

KEY CORRECTION

await does not block Flutter's UI loop; it pauses only the current async function until its Future completes.

VISUAL LEARNING MAP · READ THE FLOW BEFORE THE CODE

Flutter · Async Programming

Flutter apps frequently wait for network responses, databases, files, timers, and platform services. These operations do not finish immediately. Proper asynchronous APIs let the current function suspend at an await point while the isolate remains available to process other events, including user input and scheduled frames.

IMPORTANT CORRECTION

- Asynchronous I/O provides concurrency; it does not automatically create CPU parallelism.
- For truly CPU-intensive work, move computation to another isolate when practical.
- “Non-blocking” refers to the waiting strategy; synchronous heavy computation can still cause jank.

Operation	Why it waits	Typical API
Network request	Server and network latency	<code>http.get()</code> , <code>http.post()</code>
Database operation	Disk/query/service latency	<code>db.query()</code> , <code>db.insert()</code>
File operation	Operating-system I/O	<code>File.readAsString()</code> , <code>writeAsString()</code>
Timer / delay	Intentionally scheduled later	<code>Future.delayed()</code> , <code>Timer</code>
Platform service	Native OS/service response	plugin or platform-channel API
CPU-heavy parsing	Computation, not I/O waiting	Move to an isolate when it can cause jank

DECISION RULE

Use `async/await` for asynchronous APIs that already return Futures. Use an isolate for expensive computation when keeping that computation on the UI isolate would delay frames.

3. Synchronous vs Asynchronous Execution

Source-code order is not always completion order

Synchronous code completes the current call before moving to the next statement. Future-based work can be scheduled for later; the caller continues, and the completion runs when its Future becomes ready.

CODE

```
void main() {  
  method1();  
  method2();  
  method3();  
}  
  
void method1() => print('Method 1');  
void method2() => print('Method 2');  
void method3() => print('Method 3');
```

EXPECTED OUTPUT / STATE

```
Method 1  
Method 2  
Method 3
```

Synchronous execution order

Console

```
Method 1  
Method 2  
Method 3
```

Synchronous completion preserves the direct call order.

CODE

```
void main() {  
  method1();  
  method2();  
  method3();  
}  
  
void method1() => print('Method 1');  
void method2() {  
  Future.delayed(const Duration(seconds: 2), () {  
    print('Method 2');  
  });  
}  
void method3() => print('Method 3');
```

EXPECTED OUTPUT / STATE

```
Immediately:  
Method 1  
Method 3
```

```
After ~2 seconds:  
Method 2
```

Asynchronous completion order

Immediately

```
Method 1  
Method 3
```

After 2 seconds

```
Method 2
```

The delayed callback completes later; method3 is not blocked by the timer.

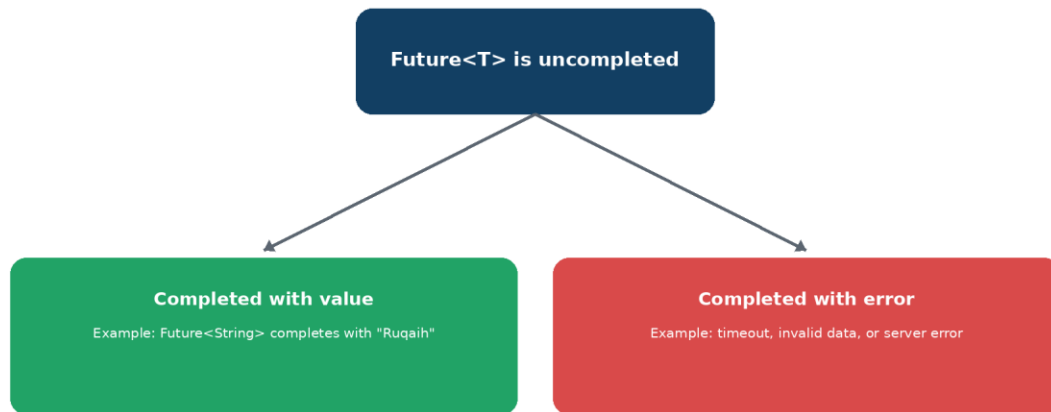
FAST UNDERSTANDING

- Scheduling asynchronous work is not the same as awaiting it.
- `Future.delayed` schedules its callback for a later event-loop turn after the delay.
- A zero-duration `Future.delayed` still runs no sooner than the next event-loop iteration, after pending microtasks.

4. Future<T>: One Eventual Result

A Future is incomplete now; later it completes exactly once with a value or an error

Future<T> completion states



A `Future<T>` represents a result that may not exist yet. Its public contract is not “poll this object for a status”; instead, consume the eventual completion with `await` or callbacks.

CODE

```

Future<String> fetchName({bool fail = false}) {
  return Future.delayed(const Duration(seconds: 2), () {
    if (fail) throw Exception('request failed');
    return 'Ruqaih Salman';
  });
}

void main() {
  final Future<String> futureName = fetchName();
  print(futureName);
}
  
```

EXPECTED OUTPUT / STATE

A `Future<String>` object is printed, not the completed String.

The exact `toString()` representation is not the learning contract.

Callbacks: `then()`, `catchError()`, `whenComplete()`

CODE

```

fetchName(fail: true)
  .then((name) => print('Name: $name'))
  .catchError((Object error) {
    print('Error: $error');
  })
  .whenComplete(() => print('Finished'));
  
```

EXPECTED OUTPUT / STATE

Error: Exception: request failed
Finished

`then()` is skipped because the `Future` completed with an error.

Method	Runs when	What it receives / does
then	Future completes with a value	Receives the successful value; returns a new <code>Future</code> .
catchError	Future completes with an error	Handles an error in the <code>Future</code> chain.
whenComplete	After value or error	Runs cleanup-like work; analogous to <code>finally</code> .

READABILITY RULE

For multi-step application logic, `async/await` is usually easier to read. Callback APIs remain useful for chaining, transformations, and understanding how `Future` completion propagates.

5. async and await

Readable top-to-bottom sequencing without blocking the whole app

Adding `async` makes the function return a `Future`. `await` suspends only the continuation of that `async` function until the awaited value becomes available; other event-loop work can continue.

CODE

```
Future<String> fetchName() async {
  await Future.delayed(const Duration(seconds: 2));
  return 'Ruqaih Salman';
}

Future<void> main() async {
  print('Before request');
  final String name = await fetchName();
  print('Name: $name');
  print('After request');
}
```

EXPECTED OUTPUT / STATE

```
Before request

[~2 seconds later]
Name: Ruqaih Salman
After request
```

await restores the expected sequence

Console

```
Before request

Name: Ruqaih Salman

After request
```

Only statements executed by the shown code appear in this output.

RULES TO REMEMBER

- `await` is valid inside an `async` or `async*` function.
- An `async` function returns a `Future`; returning `T` from its body completes `Future<T>` with that value.
- Use `Future<void>` for asynchronous work that has no meaningful result.
- A function may return a `Future` without being marked `async` if it does not need `await`.

Error handling: try / catch / finally

ERROR-HANDLING PATTERN

```
import 'dart:async';

Future<void> loadProfile() async {
  try {
    final name = await fetchName()
      .timeout(const Duration(seconds: 5));
    print(name);
  } on TimeoutException {
    print('The request timed out');
  } catch (error, stackTrace) {
    print('Unexpected error: $error');
    print(stackTrace);
  } finally {
    print('Loading finished');
  }
}
```

TYPED CATCHES

Use `on SomeException` when you can handle a known failure differently. Keep a final catch for unexpected exceptions only when you have a meaningful recovery or reporting path.

6. Timeout, Retry, and Concurrent Work

Reliability requires precise semantics, not just extra keywords

6.1 timeout(): a deadline wrapper, not cancellation

`Future.timeout` creates a new `Future` that uses the source result only if it arrives within the limit. If the timeout wins, the source operation can still continue and complete later; timeout by itself does not cancel the underlying work.

```
CODE
try {
  final value = await fetchName()
    .timeout(const Duration(milliseconds: 500));
  print(value);
} on TimeoutException {
  print('Timed out');
}
```

EXPECTED OUTPUT / STATE

Timed out

The original `fetchName()` may still finish later; its late result is not used by the timeout `Future`.

6.2 Retry with a bounded policy

REUSABLE RETRY HELPER

```
Future<T> retry<T>(<
  Future<T> Function() operation, {
    int maxAttempts = 3,
  }) async {
  for (var attempt = 1; attempt <= maxAttempts; attempt++) {
    try {
      return await operation();
    } catch (_) {
      if (attempt == maxAttempts) rethrow;
      await Future.delayed(
        Duration(milliseconds: 300 * attempt),
      );
    }
  }
  throw StateError('Unreachable');
}
```

RETRY DISCIPLINE

Retry transient failures only. Do not blindly retry validation errors, authentication failures, or non-idempotent operations that could duplicate a side effect.

6.3 Sequential await vs Future.wait

SEQUENTIAL VS CONCURRENT

```
Future<String> task(String label, int seconds) async {
  await Future.delayed(Duration(seconds: seconds));
  return label;
}

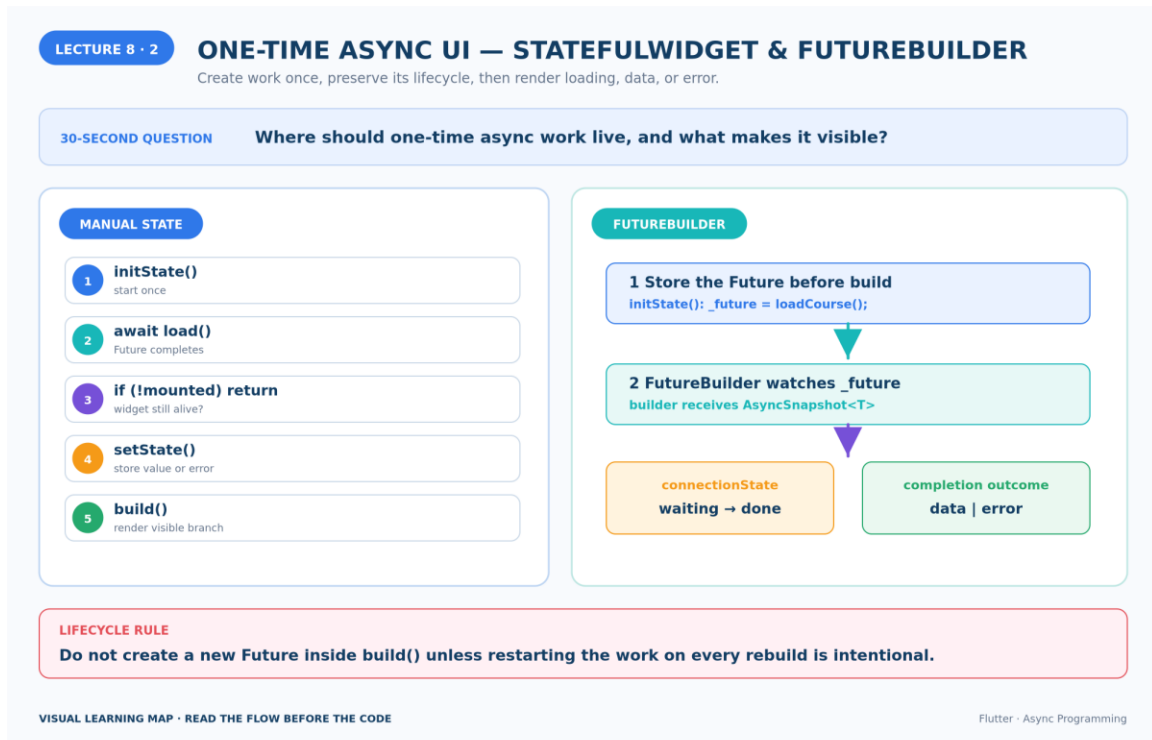
Future<void> sequential() async {
  final a = await task('A', 2);
  final b = await task('B', 1);
  print([a, b]);
}

Future<void> concurrent() async {
  final values = await Future.wait<String>([
    task('A', 2),
    task('B', 1),
  ]);
  print(values);
}
```

Question	Sequential await	Future.wait
When to use	B depends on A, or order/side effects matter.	Tasks are independent and can overlap.
Approx. demo time	~3 seconds for the shown delays.	~2 seconds for the shown delays.
Result order	Natural code order.	List preserves the input <code>Future</code> order, not completion order.
Error behavior	First awaited failure stops this sequence.	If any <code>Future</code> fails, the returned <code>Future</code> fails; default <code>eagerError</code> is <code>false</code> .

7. StatefulWidget Loading and Lifecycle Safety

Make waiting, success, error, and user actions visible—and safe



A `StatefulWidget` can start asynchronous work in `initState()`, keep the current UI state in fields, and call `setState()` when the operation finishes. After an asynchronous gap, the `State` might already have been disposed.

FOCUSED STATEFULWIDGET PATTERN

```
class ManualFutureScreen extends StatefulWidget {
  const ManualFutureScreen({super.key});

  @override
  State<ManualFutureScreen> createState() =>
    _ManualFutureScreenState();
}

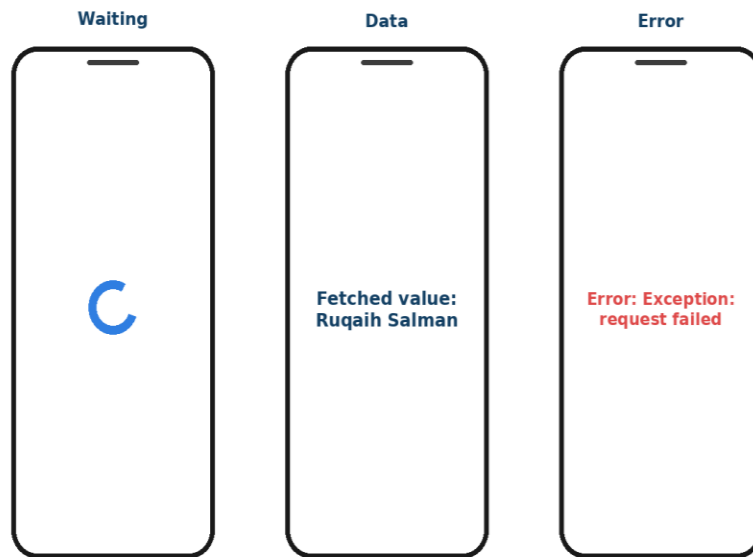
class _ManualFutureScreenState extends State<ManualFutureScreen> {
  String? value;
  Object? error;

  @override
  void initState() {
    super.initState();
    _load();
  }

  Future<void> _load() async {
    try {
      final result = await fetchName();
      if (!mounted) return;
      setState(() => value = result);
    } catch (e) {
      if (!mounted) return;
      setState(() => error = e);
    }
  }

  @override
  Widget build(BuildContext context) {
    if (error != null) return Center(child: Text('Error: $error'));
    if (value == null) {
      return const Center(child: CircularProgressIndicator());
    }
    return Center(child: Text('Fetched value: $value'));
  }
}
```

Manual StatefulWidget visible branches



The displayed branches correspond to waiting, successful data, and error.

Prevent duplicate actions

```
DUPLICATE-ACTION GUARD
bool _saving = false;

Future<void> save() async {
  if (_saving) return;
  setState(() => _saving = true);
  try {
    await saveToServer();
    if (!mounted) return;
    setState(() => message = 'Saved');
  } finally {
    if (mounted) setState(() => _saving = false);
  }
}

// In build():
// onPressed: _saving ? null : save,
```

BUILDCONTEXT AFTER AWAIT

Before using Navigator, ScaffoldMessenger, or another BuildContext-dependent API after an async gap, check `context.mounted` when the context may no longer be valid.

8. FutureBuilder in Depth

A declarative widget for one Future and its AsyncSnapshot

FutureBuilder rebuilds from the latest AsyncSnapshot of a Future. The Future should be obtained before build—commonly in initState(), didUpdateWidget(), or didChangeDependencies()—not created as a side effect of building the widget.

Property / state	Meaning
future	The Future<T> being observed.
builder	Returns the widget for the current AsyncSnapshot<T>.
initialData	Optional data available before the Future completes.
ConnectionState.none	No Future is connected, or a transition while replacing a previously completed Future.
ConnectionState.waiting	A non-null Future is connected and has not completed.
ConnectionState.done	The Future completed with data or an error.
snapshot.hasData / data	Successful non-null data.
snapshot.hasError / error	Completed error.

PRECISION NOTE

FutureBuilder normally transitions through waiting and done for a connected Future; ConnectionState.active is associated with StreamBuilder interaction, not normal Future completion.

DO NOT CREATE THE FUTURE INSIDE BUILD

- Every parent rebuild can create a new Future and restart the asynchronous task.
- Create/store the Future before build and replace it only for an intentional refresh/retry or a changed dependency.
- Assume build may run frequently; avoid side effects inside it.

STORED FUTURE + RETRY

```
class NameScreen extends StatefulWidget {
  const NameScreen({super.key});

  @override
  State<NameScreen> createState() => _NameScreenState();
}

class _NameScreenState extends State<NameScreen> {
  late Future<String> _nameFuture;

  @override
  void initState() {
    super.initState();
    _nameFuture = fetchName(fail: true);
  }

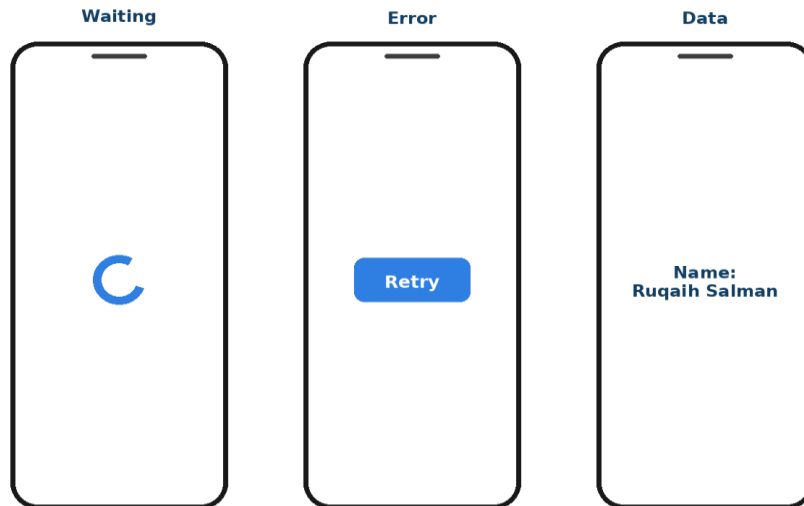
  void retry() {
    setState(() => _nameFuture = fetchName());
  }

  @override
  Widget build(BuildContext context) {
    return FutureBuilder<String>(
      future: _nameFuture,
      builder: (context, snapshot) {
        if (snapshot.connectionState != ConnectionState.done) {
          return const CircularProgressIndicator();
        }
        if (snapshot.hasError) {
          return ElevatedButton(
            onPressed: retry,
            child: const Text('Retry'),
          );
        }
        return Text('Name: ${snapshot.requireData}');
      },
    );
  }
}
```

8.1. FutureBuilder: Exact Visible States, Refresh, and Retry

Keep the code and the displayed output consistent

FutureBuilder exact states



The example below intentionally fails on the first load so the error branch is reachable. Retry replaces the stored Future with a successful one. Refresh after success starts another Future. The builder treats both none and waiting as a loading state by checking for done.

COMPLETE RUNNABLE FUTUREBUILDER EXAMPLE

```
import 'package:flutter/material.dart';

void main() => runApp(const MaterialApp(home: ProfileScreen()));

Future<String> fetchProfileName({bool fail = false}) async {
  await Future.delayed(const Duration(seconds: 2));
  if (fail) throw Exception('Server unavailable');
  return 'Ruqaih Salman';
}

class ProfileScreen extends StatefulWidget {
  const ProfileScreen({super.key});

  @override
  State<ProfileScreen> createState() => _ProfileScreenState();
}

class _ProfileScreenState extends State<ProfileScreen> {
  late Future<String> _future;

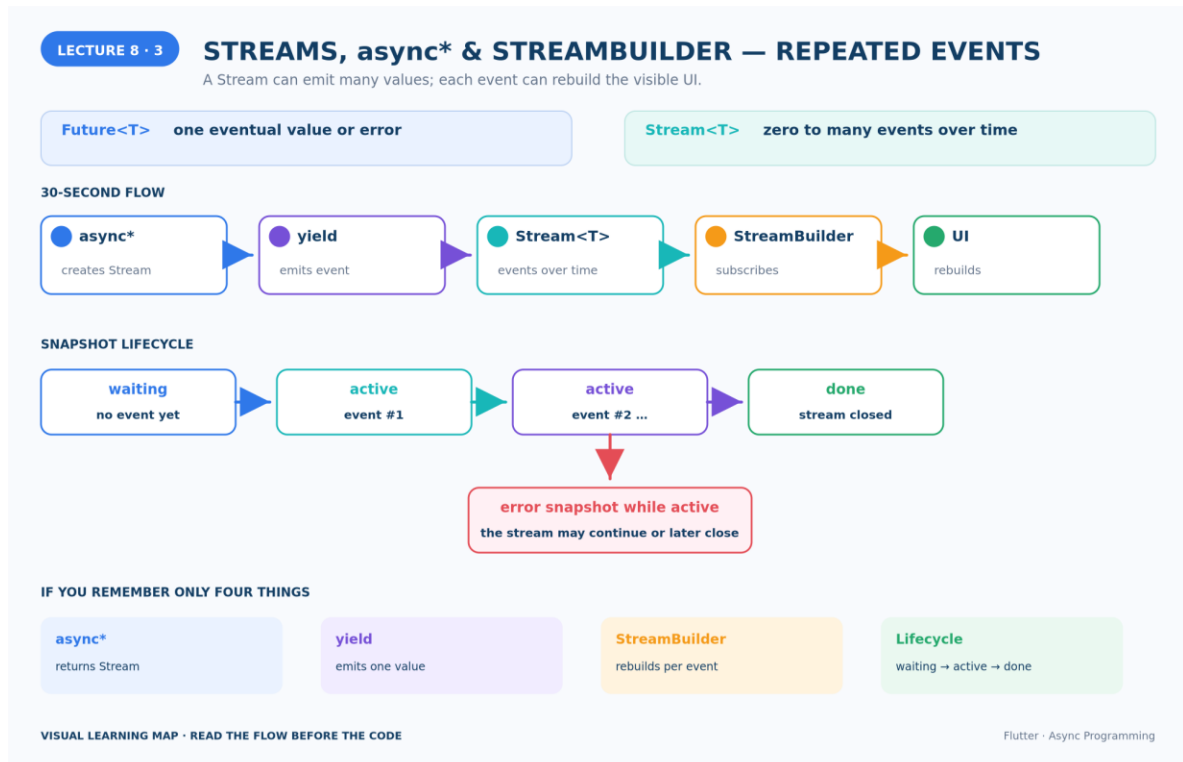
  @override
  void initState() {
    super.initState();
    _future = fetchProfileName(fail: true); // error demo
  }

  void reload() {
    setState(() => _future = fetchProfileName());
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Async profile')),
      body: Center(
        child: FutureBuilder<String>(
          future: _future,
          builder: (context, snapshot) {
            if (snapshot.connectionState != ConnectionState.done) {
              return const CircularProgressIndicator();
            }
            if (snapshot.hasError) {
              return Column(
                mainAxisAlignment: MainAxisAlignment.min,
                children: [
                  Text('Error: ${snapshot.error}'),
                  const SizedBox(height: 12),
                  ElevatedButton(onPressed: reload, child: const Text('Retry')),
                ],
              );
            }
            return Column(
              mainAxisAlignment: MainAxisAlignment.min,
              children: [
                Text('Welcome ${snapshot.requireData}'),
                const SizedBox(height: 12),
                OutlinedButton(onPressed: reload, child: const Text('Refresh')),
              ],
            );
          },
        ),
      ),
    );
  }
}
```

9. Streams, async*, and yield

Use a Stream when values may arrive repeatedly



A Future completes once. A Stream can emit zero, one, or many data events over time, can emit errors, and eventually can close. Typical examples include clocks, sensor readings, sockets, progress events, and database change notifications.

Future<T>	Stream<T>
One eventual value or error.	Zero, one, or many events over time, plus possible errors.
Consume with await, then, or FutureBuilder.	Consume with await for, listen, or StreamBuilder.
Completes once.	Emits repeatedly and can then close.

async* and yield

CODE

```

Stream<int> counterStream() async* {
  for (var value = 1; value <= 5; value++) {
    await Future.delayed(const Duration(seconds: 1));
    yield value;
  }
}

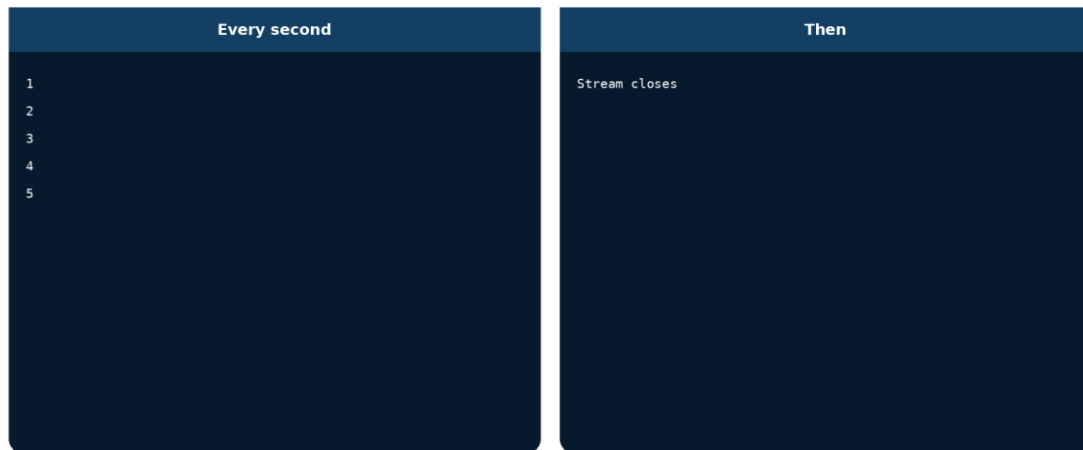
Future<void> main() async {
  await for (final value in counterStream()) {
    print(value);
  }
  print('Stream closes');
}
  
```

EXPECTED OUTPUT / STATE

```

1
2
3
4
5
Stream closes
  
```

A Stream emits repeated values



Each yield adds a new event; await for finishes when the stream closes.

SINGLE-SUBSCRIPTION DETAIL

A stream produced by an `async*` function is a single-subscription stream, and each call to the `async*` function creates a new stream instance.

9.1. Stream Subscriptions and Cancellation

Manual ownership means manual lifecycle management

Calling `stream.listen()` returns a `StreamSubscription`. If your `State` owns that subscription, store it and cancel it in `dispose()` so it stops receiving events and can release resources.

MANUALLY OWNED SUBSCRIPTION

```
import 'dart:async';
import 'package:flutter/material.dart';

class SensorScreen extends StatefulWidget {
  const SensorScreen({super.key});

  @override
  State<SensorScreen> createState() => _SensorScreenState();
}

class _SensorScreenState extends State<SensorScreen> {
  StreamSubscription<int>? _subscription;
  int? latest;

  @override
  void initState() {
    super.initState();
    _subscription = counterStream().listen((value) {
      if (!mounted) return;
      setState(() => latest = value);
    });
  }

  @override
  void dispose() {
    _subscription?.cancel();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Text('Latest: ${latest ?? '-'}');
  }
}
```

Subscription action	Meaning
<code>listen(...)</code>	Creates a <code>StreamSubscription</code> and starts receiving events.
<code>pause() / resume()</code>	Temporarily pauses/resumes event delivery according to stream behavior.
<code>cancel()</code>	Stops receiving events and returns a <code>Future</code> that completes after cleanup.
<code>cancelOnError: true</code>	Automatically cancels after the first delivered error event.

10. StreamBuilder in Depth

Build the UI from the latest stream event

Argument/ state	Meaning
stream	The Stream<T> being observed.
initialData	Optional value for the first build before an event arrives.
builder	Builds a widget from the latest AsyncSnapshot<T>.
ConnectionState.waiting	Subscribed, but no event has been delivered yet.
ConnectionState.active	One or more events are being delivered.
ConnectionState.done	The stream has closed.
snapshot.hasError	The latest stream interaction contains an error.

As with FutureBuilder, the stream should usually be obtained before build. Creating a new stream during every build can restart the asynchronous source when the widget rebuilds.

```

STORED REAL-TIME CLOCK STREAM
class ClockScreen extends StatefulWidget {
  const ClockScreen({super.key});

  @override
  State<ClockScreen> createState() => _ClockScreenState();
}

class _ClockScreenState extends State<ClockScreen> {
  late final Stream<DateTime> _clock;

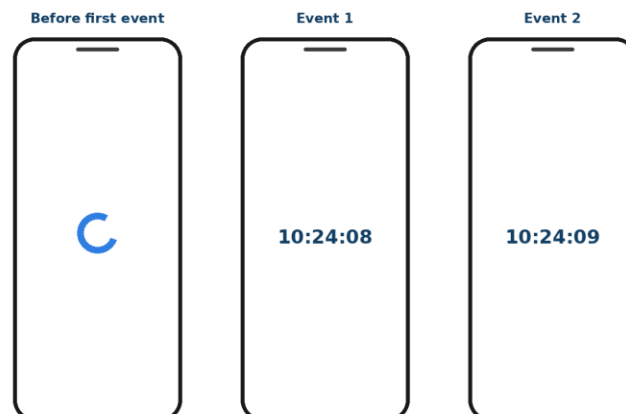
  @override
  void initState() {
    super.initState();
    _clock = clockStream();
  }

  Stream<DateTime> clockStream() async* {
    while (true) {
      yield DateTime.now();
      await Future.delayed(const Duration(seconds: 1));
    }
  }

  @override
  Widget build(BuildContext context) {
    return Center(
      child: StreamBuilder<DateTime>(
        stream: _clock,
        builder: (context, snapshot) {
          if (snapshot.hasError) {
            return Text('Error: ${snapshot.error}');
          }
          if (!snapshot.hasData) {
            return const CircularProgressIndicator();
          }
          final time = snapshot.requireData;
          return Text(
            '${time.hour.toString().padLeft(2, '0')}: '
            '${time.minute.toString().padLeft(2, '0')}: '
            '${time.second.toString().padLeft(2, '0')}',
          );
        },
      ),
    );
  }
}

```

StreamBuilder real-time clock



Before the first event a loading state may appear; later events update the latest time.

OWNERSHIP RULE • StreamBuilder manages its own subscription. Manual stream.listen() requires storing the StreamSubscription and cancelling it in dispose().

11. Complete Practical Example: FutureBuilder App

Loading → error → retry → data → refresh, with no Future created inside build

This is the consolidated runnable Future example used throughout the lecture. It intentionally demonstrates both error and success paths and keeps Future ownership explicit.

OUTPUT CONVENTION Console Output = text printed by the program. Expected UI Output = an illustrative screen state derived from the displayed code. Actual Emulator Output is used only after a real emulator/device run.

```
FULL APP — PROFILE FUTURE
import 'package:flutter/material.dart';

void main() => runApp(
  MaterialApp(
    theme: ThemeData(useMaterial3: true, colorSchemeSeed: Colors.indigo),
    home: const ProfileScreen(),
  ),
);

Future<String> fetchProfileName({bool fail = false}) async {
  await Future.delayed(const Duration(seconds: 2));
  if (fail) throw Exception('Server unavailable');
  return 'Ruqaih Salman';
}

class ProfileScreen extends StatefulWidget {
  const ProfileScreen({super.key});
  @override
  State<ProfileScreen> createState() => _ProfileScreenState();
}

class _ProfileScreenState extends State<ProfileScreen> {
  late Future<String> _future;

  @override
  void initState() {
    super.initState();
    _future = fetchProfileName(fail: true); // error demo
  }

  void reload() => setState(() => _future = fetchProfileName());

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Async profile')),
      body: Center(
        child: FutureBuilder<String>(
          future: _future,
          builder: (context, snapshot) {
            if (snapshot.connectionState != ConnectionState.done) {
              return const CircularProgressIndicator();
            }
            if (snapshot.hasError) {
              return Column(
                mainAxisAlignment: MainAxisAlignment.min,
                children: [
                  Text('Error: ${snapshot.error}'),
                  const SizedBox(height: 12),
                  ElevatedButton(onPressed: reload, child: const Text('Retry')),
                ],
              );
            }
            return Column(
              mainAxisAlignment: MainAxisAlignment.min,
              children: [
                Text('Welcome ${snapshot.requireData}'),
                const SizedBox(height: 12),
                OutlinedButton(onPressed: reload, child: const Text('Refresh')),
              ],
            );
          },
        ),
      ),
    );
  }
}
```

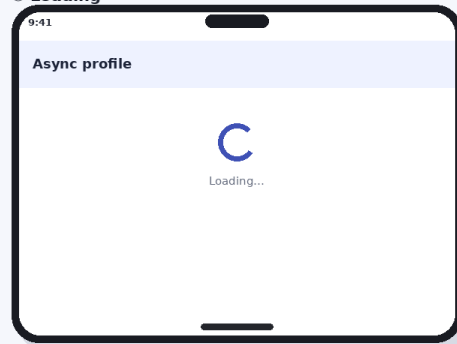
WHAT THIS APP DEMONSTRATES

- Future<T> returning either a value or an exception.
- Future stored in State before build.
- FutureBuilder loading, error, and data rendering.
- Intentional replacement of the Future for retry/refresh.
- No asynchronous side effect created by build().

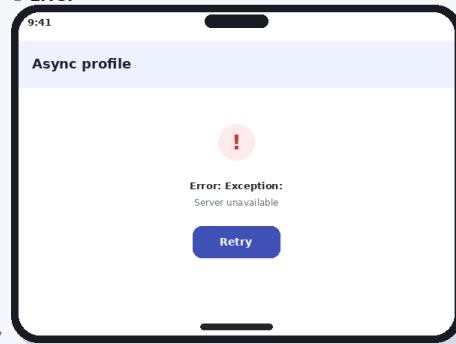
Expected UI Output — FutureBuilder

Illustrative screen states derived from the displayed code (not an emulator screenshot).

① Loading

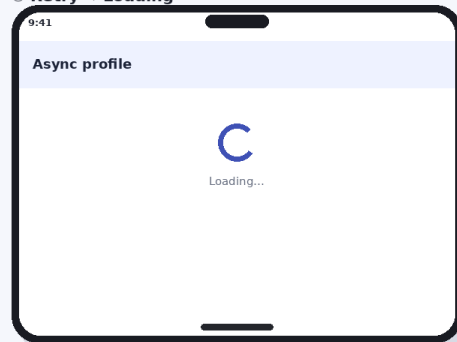


② Error

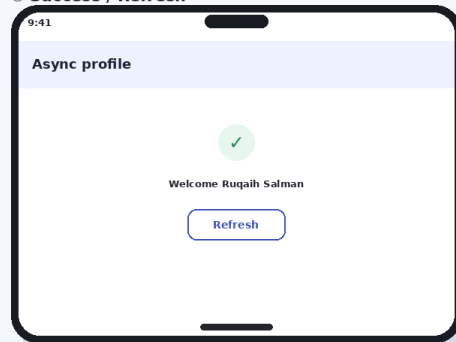


Retry
replaces the stored Future

③ Retry → Loading



④ Success / Refresh



Expected UI Output — FutureBuilder state sequence

Illustrative output derived from the displayed code; it is not an emulator screenshot.

State path: initial Future → waiting → error → Retry replaces the Future → waiting → data → Refresh starts a new Future.

User flow	Reachable visible state
Initial build	Loading spinner → error message + Retry.
Tap Retry	Loading spinner → Welcome Ruqaih Salman + Refresh.
Tap Refresh	Loading spinner → successful data again.
Timing contract	Builder timing is not guaranteed to expose every intermediate snapshot; render every reachable state correctly.

11.1. Complete Practical Example: StreamBuilder App

A complete real-time clock with a stored Stream, plus representative expected UI states

```

FULL APP — REAL-TIME CLOCK
import 'package:flutter/material.dart';

void main() => runApp(
  MaterialApp(
    theme: ThemeData(useMaterial3: true, colorSchemeSeed: Colors.indigo),
    home: const ClockScreen(),
  ),
);

class ClockScreen extends StatefulWidget {
  const ClockScreen({super.key});
  @override
  State<ClockScreen> createState() => _ClockScreenState();
}

class _ClockScreenState extends State<ClockScreen> {
  late final Stream<DateTime> _clock;

  @override
  void initState() {
    super.initState();
    _clock = _clockStream();
  }

  Stream<DateTime> _clockStream() async* {
    while (true) {
      yield DateTime.now();
      await Future.delayed(const Duration(seconds: 1));
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Real-time clock')),
      body: Center(
        child: StreamBuilder<DateTime>(
          stream: _clock,
          builder: (context, snapshot) {
            if (snapshot.hasError) {
              return Text('Error: ${snapshot.error}');
            }
            if (!snapshot.hasData) {
              return const CircularProgressIndicator();
            }
            final time = snapshot.requireData;
            final hh = time.hour.toString().padLeft(2, '0');
            final mm = time.minute.toString().padLeft(2, '0');
            final ss = time.second.toString().padLeft(2, '0');
            return Text(
              '$hh:$mm:$ss',
              style: Theme.of(context).textTheme.headlineMedium,
            );
          },
        ),
      ),
    );
  }
}

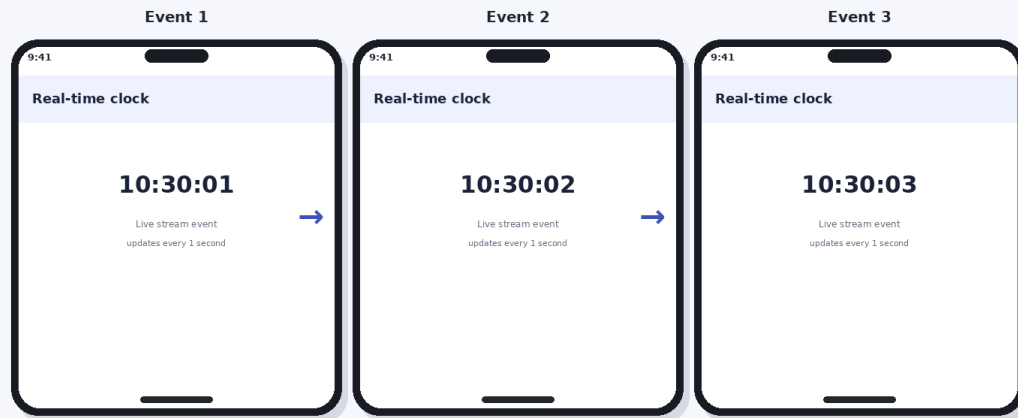
```

WHAT THIS APP DEMONSTRATES

- `async*` creates a stream and `yield` emits each new `DateTime`.
- The `Stream` is stored once in `initState` instead of recreated by `build`.
- `StreamBuilder` owns the listener and rebuilds from the latest snapshot.
- The UI has explicit loading, error, and data branches.

Expected UI Output — StreamBuilder Clock

Three representative frames show repeated Stream events over time.



Expected UI Output — StreamBuilder real-time clock

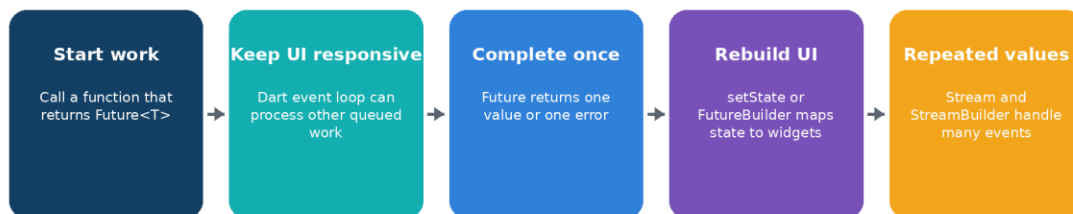
Illustrative output derived from the displayed code; the actual time values depend on the moment of execution.

State path: stream subscription → first `DateTime` event → `ConnectionState.active` → a new event every second → the UI rebuilds with the latest value.

12. Integrated Async UI Checklist

A compact diagnostic model for real Flutter screens

Async programming: the complete learning path



Step	Question to ask	Typical failure if ignored
1. Start	What starts the operation? Is it intentional?	Repeated network calls or duplicate taps.
2. Future / Stream	Is this one result or repeated events?	Using the wrong abstraction.
3. Wait	Which function is suspended at await?	Confusing function suspension with app blocking.
4. Error	What exceptions/timeouts are expected?	Silent failure or uncaught errors.
5. Lifecycle	Can State/BuildContext be disposed before completion?	setState/Navigator after dispose.
6. Rebuild	Where is the Future/Stream created and replaced?	Restarting work on every build.
7. Visible state	Can the user see loading, data, error, and retry?	Frozen-looking or ambiguous UI.
8. Cleanup	Who owns subscriptions/resources?	Leaking listeners or duplicate callbacks.

High-yield distinctions

Do not confuse	Correct distinction
Asynchronous v s parallel	Async I/O allows concurrency on the event loop; CPU parallelism requires additional isolates/cores.
Future v s final value	Calling an async function gives a Future now; await obtains its eventual value/error.
timeout v s cancellation	timeout chooses an alternate Future outcome; the source can continue unless separately cancelable.

Do not confuse	Correct distinction
FutureBuilder vs Future creation	FutureBuilder observes a Future; it should not usually create that Future in build.
StreamBuilder vs manual listen	StreamBuilder manages its subscription; manual listen requires explicit subscription lifecycle.
waiting vs active	FutureBuilder normally uses waiting/done; StreamBuilder uses waiting/active/done as events arrive.

ONE-SENTENCE MENTAL MODEL

Start asynchronous work, keep the UI responsive while waiting, resume with a value or error, verify lifecycle validity, then rebuild a visible state.

12.1. Assessment

Check understanding before copying patterns into another app

Task	Question
Predict	In the Future.delayed method1/method2/method3 example, what prints immediately and what prints later? Why?
Explain	Why does printing fetchName() directly not print the final name?
Differentiate	When should you choose FutureBuilder instead of StreamBuilder?
Fix	Why should a Future usually be created before build instead of by calling fetchData() inside build?
Lifecycle	Why can setState() fail after await, and what does mounted protect?
Concurrency	When is Future.wait better than sequential await? What order does its result list use?
Reliability	Why is Future.timeout not the same as cancelling the original operation?
Streams	If you call stream.listen() manually inside State, what object should you store and what should you do in dispose()?
Debug	A button can be tapped five times while a request is running. Give two UI/code techniques to prevent duplicate work.
Design	Choose the correct abstraction for: one database query, download progress, a sensor, and a one-time profile load.

EXAM TRAPS

- "async means parallel" — false.
- "await blocks the whole UI thread" — false; it suspends the current async continuation.
- "FutureBuilder should call the API inside build" — usually false.
- "timeout cancels the original Future" — false.
- "StreamBuilder requires manual cancel in dispose" — false for its own internal subscription; manual listen does.

12.2. Answer Key and Official References

Concise model answers + verified documentation baseline

#	Model answer
1	Method 1 and Method 3 print immediately; Method 2 prints after the delay because only its callback was scheduled for later.
2	fetchName() returns Future<String> immediately. await (or a callback) consumes the eventual String/error.
3	Use FutureBuilder for one eventual result; use StreamBuilder for repeated events/progress over time.
4	Creating a Future inside build can restart the task on rebuild. Store it before build and replace it intentionally.
5	State may be disposed while awaiting. mounted/context.mounted confirms the State/BuildContext is still valid before UI access.
6	Use Future.wait for independent Futures that can overlap. Its result list follows input order, not completion order.
7	timeout completes a wrapper Future with a timeout outcome; the source Future can continue and finish later.
8	Store StreamSubscription<T>; cancel it in dispose() when you own the manual subscription.
9	Disable the action while loading and/or guard the handler with a boolean/request-in-progress check.
10	One DB query → Future/FutureBuilder; progress → Stream/StreamBuilder; sensor → Stream/StreamBuilder; profile load → Future/FutureBuilder.

COMPATIBILITY BASELINE

Verified against official documentation available on 11 August 2026. Flutter documentation reflects Flutter 3.44.7; Dart documentation/API reflects Dart 3.12.2. The lecture uses APIs and patterns valid for that stable baseline.

Official references

- Dart asynchronous programming — <https://dart.dev/language/async>
- Dart concurrency and isolates — <https://dart.dev/language/concurrency>
- Dart Future API — <https://api.dart.dev/dart-async/Future-class.html>
- Dart Future.wait — <https://api.dart.dev/dart-async/Future/wait.html>
- Dart Future.timeout — <https://api.dart.dev/dart-async/Future/timeout.html>
- Dart Stream / StreamSubscription — <https://api.dart.dev/dart-async/Stream-class.html>
- Flutter FutureBuilder — <https://api.flutter.dev/flutter/widgets/FutureBuilder-class.html>
- Flutter StreamBuilder — <https://api.flutter.dev/flutter/widgets/StreamBuilder-class.html>
- Flutter State.mounted — <https://api.flutter.dev/flutter/widgets/State/mounted.html>
- Flutter SDK archive / release baseline — <https://docs.flutter.dev/install/archive>
- Dart SDK archive / documentation baseline — <https://dart.dev/get-dart/archive>

FINAL NOTE

Every code example in this edition is either complete and runnable as shown or explicitly labeled as a focused/reusable snippet. Output panels describe only states reachable from the displayed code.